

★ Logistic Regression

logistic regression is mainly a classification algorithm, not a regression algorithm.

1. Why can't we simply use linear regression?

Suppose we want to predict whether a student will pass: $Y = \begin{cases} 1 & \text{Pass} \\ 0 & \text{Fail} \end{cases}$

Linear regression would produce: $\hat{y} = \beta_0 + \beta_1 x$

Suppose the model learns: $\hat{y} = -0.4 + 0.3(\text{study hours})$

For a student studying 1 hour: $\hat{y} = -0.4 + 0.3(1) = -0.1$

For a student studying 6 hours: $\hat{y} = -0.4 + 0.3(6) = 1.4$

★ But probability must remain between 0 and 1.

Linear regression may predict: -0.1, 1.4, 2.2

These cannot be probabilities.

Logistic regression solves this problem by passing the linear result through a function that always produces a value between 0 and 1.

2. The logistic regression model

Logistic regression first creates a linear score: $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m$

★ Then it passes z through the sigmoid function:

$$p = \frac{1}{1+e^{-z}}$$

Here:

- z can have any value from $-\infty$ to $+\infty$
- p will always be between 0 and 1
- p represents the predicted probability that $Y = 1$

Therefore:

$$P(Y = 1 | X) = \frac{1}{1+e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_m x_m)}}$$

How sigmoid converts scores into probabilities

Linear score z Probability p

-5 0.0067

-2 0.1192

0 0.5000

1 0.7311

2 0.8808

5 0.9933

Notice:

- Large negative $z \rightarrow$ probability close to 0
- $z = 0 \rightarrow$ probability exactly 0.5
- Large positive $z \rightarrow$ probability close to 1

3. prediction example

Suppose the model has learned: $z = -4 + 1.2(\text{study hours})$

Therefore: $p = \frac{1}{1+e^{-z}}$

Student studying 2 hours $z = -4 + 1.2(2) = -1.6$

$$p = \frac{1}{1+e^{1.6}} = 0.168$$

The predicted probability of passing is approximately 16.8%.

Using the usual threshold of 0.5: $0.168 < 0.5$

Therefore: Predicted class=0=Fail

★ **Student studying 5 hours** $z = -4 + 1.2(5) = 2$

$$p = \frac{1}{1+e^{-2}} = 0.881$$

The predicted probability of passing is 88.1%.

Since: $0.881 \geq 0.5$

the predicted class is: $1 = \text{Pass}$

Probability and classification are different

Logistic regression first gives a probability: $p = 0.881$

Then we apply a threshold:

$$\hat{y} = \begin{cases} 1 & p \geq 0.5 \\ 0 & p < 0.5 \end{cases}$$

★ The threshold does not always have to be 0.5. For medical screening, we might use 0.2 to avoid missing high-risk patients.

Why is it called “logistic regression”?

Although the final purpose is classification, the model creates a linear regression-like equation for the log-odds.

Start with: $p = \frac{1}{1+e^{-z}}$

This can be rearranged to: $\frac{p}{1-p} = e^z$

Taking the natural logarithm: $\log\left(\frac{p}{1-p}\right) = z$

Since: $z = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m$

★

we get: $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_m x_m$

Therefore, logistic regression is:

- nonlinear in probability p
- linear in the log-odds

Probability, odds and log-odds

Suppose: $p = 0.80$

The odds are:

★

$$\text{odds} = \frac{p}{1-p} = \frac{0.80}{0.20} = 4$$

This means the event is four times as likely to happen as not happen.

Or among 100 similar students, we might expect approximately:

- 80 to pass;
- 20 not to pass.

In terms of odds, $\text{Odds in favour} = 4:1$

In terms of students:

- approximately 4 students pass;
- for every 1 student who does not pass.

The log-odds are: $\log(4) = 1.386$

The value 1.386 is not directly a percentage. It is simply the odds of 4 expressed on a logarithmic scale.

To interpret it, reverse the logarithm: $e^{1.386} = 4$

Then convert odds into probability: $p = \frac{\text{odds}}{1+\text{odds}}$

$$p = \frac{4}{1+4} = 0.80$$

Therefore: $\log\text{-odds} = 1.386 \Rightarrow \text{odds} = 4 \Rightarrow \text{probability} = 80\%$

★ Another example: $p = 0.20$

$$\text{odds} = \frac{0.20}{0.80} = 0.25$$

$$\log(0.25) = -1.386$$

The negative sign tells us: $\text{odds} < 1$

This means: The event is less likely to happen than not happen.

Probability, odds, and log-odds are three different ways of describing the same chance.

★ What do the coefficients mean?

Suppose the model is: $\log\left(\frac{p}{1-p}\right) = -4 + 0.8(\text{study hours}) + 0.03(\text{attendance})$

The coefficients are: $\beta_0 = -4, \beta_1 = 0.8, \beta_2 = 0.03$

A one-unit increase in x_j changes the log-odds by β_j , while other variables remain constant. Therefore, one additional study hour increases the log-odds of passing by 0.8.

However, odds are usually easier to explain.

Exponentiate the coefficient: $e^{0.8} = 2.23$

Therefore:

For every additional study hour, the odds of passing are multiplied by approximately 2.23, assuming attendance remains constant.

For attendance: $e^{0.03} = 1.030$

Therefore:

A one-percentage-point increase in attendance multiplies the odds of passing by approximately 1.03

A negative coefficient reduces the odds. For example: $\beta = -0.5$

$$e^{-0.5} = 0.607$$

A one-unit increase in that feature multiplies the odds by 0.607

★

cost function?

A cost function tells us: How bad are the model's current predictions?

In linear regression, we commonly use mean squared error:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Logistic regression normally uses **binary cross-entropy**, also called:

- log loss
- logistic loss
- negative log-likelihood

For one observation: $L_i = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$

For the entire dataset: $J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$

Here:

- y_i is the actual class, 0 or 1
- p_i is the predicted probability of class 1
- $J(\beta)$ is the average cost

★

Understanding the two cases

When the actual class is $y = 1$

Substitute $y = 1$:

$$L = -[1 \log(p) + (1 - 1) \log(1 - p)]$$

$$L = -\log(p)$$

Therefore, when the student actually passed, the model should give a high passing probability.

Actual y Predicted p Loss

★	1	0.99	0.010
	1	0.90	0.105
	1	0.60	0.511
	1	0.10	2.303
	1	0.01	4.605

Confident and correct predictions receive a very small loss. Confident and wrong predictions receive a very large loss.

When the actual class is $y = 0$

Substitute $y = 0$:

$$L = -[0\log(p) + (1 - 0)\log(1 - p)]$$

$$L = -\log(1 - p)$$

Actual y Predicted p Loss

★	0	0.01	0.010
	0	0.10	0.105
	0	0.40	0.511
	0	0.90	2.303
	0	0.99	4.605

Remember that p is the predicted probability of class 1.

If the actual class is 0, we want p to be close to 0.

cost-function example

Consider three students:

Student Actual y Predicted probability p

A	1	0.80
B	0	0.30

$$C \quad 1 \quad 0.60$$

$$\text{For A: } L_A = -\log(0.80) = 0.223$$

$$\text{For B: } L_B = -\log(1 - 0.30) = -\log(0.70) = 0.357$$

$$\text{For C: } L_C = -\log(0.60) = 0.511$$

Average log loss:

$$J = \frac{0.223 + 0.357 + 0.511}{3}$$

$$J = 0.364$$

The training algorithm changes the coefficients to make this cost smaller.

Where does log loss come from?

This comes from the likelihood function.

For one observation:

★

$$P(Y = y_i | X_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

This compact formula handles both cases.

If $y_i = 1$:

$$p_i^1 (1 - p_i)^0 = p_i$$

If $y_i = 0$:

★

$$p_i^0 (1 - p_i)^1 = 1 - p_i$$

Assuming observations are independent, the joint likelihood is:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

We want coefficients that maximize this likelihood.

Instead of maximizing a product, we take the logarithm:

$$\log L(\beta) = \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

The logarithm changes products into sums, which are easier and more numerically stable to optimize.

★ Therefore: maximize log-likelihood is equivalent to: minimize negative log-likelihood

And negative average log-likelihood is: log loss

So these ideas are connected:

Log loss = negative log-likelihood = binary cross-entropy

They may differ only in whether the loss is summed or averaged.

How are coefficients learned?

The model does not know the correct values of: $\beta_0, \beta_1, \dots, \beta_m$

It finds them through numerical optimization.

Conceptually:

1. Start with initial coefficient values.
- ★ 2. Calculate the linear scores z .
3. Convert the scores into probabilities.
4. Calculate log loss.
5. Determine how each coefficient contributed to the error.
6. update the coefficients.
7. Repeat until the cost stops decreasing meaningfully.

This is where gradient descent can be used.

gradient descent?

Gradient descent is an optimization method for finding coefficient values that minimize a cost function.

★ Imagine standing on a mountain in fog. You want to reach the lowest point.

You:

1. check which direction slopes upward;
2. move in the opposite direction;
3. repeat until the ground is almost flat.

Mathematically:

$$\beta_j^{\text{new}} = \beta_j^{\text{old}} - \alpha \frac{\partial J}{\partial \beta_j}$$

Here:

- β_j = one coefficient
- J = cost function
- $\frac{\partial J}{\partial \beta_j}$ = gradient or slope of cost with respect to that coefficient
- α = learning rate

Why do we subtract the gradient?

★

The gradient points towards increasing cost.

Therefore, to reduce the cost, we move in the opposite direction:

$$\beta_{\text{new}} = \beta_{\text{old}} - \text{movement}$$

Gradient formula for logistic regression

★

For logistic regression: $p_i = \sigma(\beta_0 + \beta_1 x_{i1} + \dots + \beta_m x_{im})$

The gradient for coefficient β_j is: $\frac{\partial J}{\partial \beta_j} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_{ij}$

The intercept can be treated as a coefficient whose feature is always 1:

$$x_{i0} = 1$$

Therefore:

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)$$

The important part is:

★

$$p_i - y_i$$

This represents probability error.

If actual $y = 1$, but predicted $p = 0.2$

$$p - y = 0.2 - 1 = -0.8$$

The negative gradient helps move the appropriate coefficients upward.

If actual $y = 0$, but predicted $p = 0.9$

$$p - y = 0.9 - 0 = 0.9$$

The positive gradient causes the update to reduce the appropriate coefficients.

gradient-descent iteration example

Suppose our training data is:

Study hours x Actual result y

0	0
1	0
2	1

★

Start with:

$$\beta_0 = 0, \beta_1 = 0$$

Therefore, for every student:

$$z = 0 + 0x = 0$$

$$p = \frac{1}{1 + e^0} = 0.5$$

Initial predictions:

★

x	y	p	$p - y$
0	0	0.5	0.5
1	0	0.5	0.5
2	1	0.5	-0.5

Gradient for intercept

$$\frac{\partial J}{\partial \beta_0} = \frac{0.5 + 0.5 - 0.5}{3} = 0.1667$$

★

Gradient for study-hours coefficient

$$\frac{\partial J}{\partial \beta_1} = \frac{(0.5)(0) + (0.5)(1) + (-0.5)(2)}{3}$$

$$= \frac{0 + 0.5 - 1}{3} = -0.1667$$

Suppose the learning rate is:

$$\alpha = 0.5$$

Update the intercept:

$$\beta_0^{\text{new}} = 0 - (0.5)(0.1667) = -0.08335$$

Update the slope:

$$\beta_1^{\text{new}} = 0 - (0.5)(-0.1667) = 0.08335$$

★

The new model is:

$$z = -0.08335 + 0.08335x$$

New probabilities are approximately:

x	Actual y	New p
0	0	0.479
1	0	0.500
2	1	0.521

The predictions have started moving in the correct direction.

The initial loss was:

$$0.693$$

After this small update, the loss is approximately:

$$0.666$$

★

Gradient descent repeats these updates until the coefficients converge.

14. Why doesn't OLS give a direct formula here?

In logistic regression:

$$p = \frac{1}{1 + e^{-x\beta}}$$

The sigmoid makes the relationship nonlinear in probability. Setting the derivatives to zero does not produce a simple closed-form solution for β .

Therefore, logistic regression coefficients are found iteratively using optimization algorithms such as:

- gradient descent
- stochastic gradient descent
- Newton's method
- L-BFGS
- SAG
- SAGA

★ How does logistic regression work for multiple classes?

Suppose the target has three flower classes:

$Y \in \{\text{Setosa, Versicolor, Virginica}\}$

There are two main strategies.

Method 1: One-vs-rest

Create one binary model for every class:

1. Setosa versus everything else
2. Versicolor versus everything else
3. Virginica versus everything else

Suppose the three models output:

Model	Probability
Setosa vs rest	0.10
Versicolor vs rest	0.70
Virginica vs rest	0.40

The selected class is Versicolor because it received the highest score.

A limitation is that these probabilities come from separately trained models and may not naturally sum to 1.

★ Method 2: Multinomial logistic regression

Multinomial logistic regression learns a separate score for every class.

For class k :

$$z_k = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{km}x_m$$

If there are three classes, we obtain:

$$z_1, z_2, z_3$$

These scores are converted into probabilities using the **softmax function**:

$$P(Y = k | X) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

All resulting probabilities:

- lie between 0 and 1;
- add up to exactly 1.

Term	Meaning
Linear score	$z = \beta_0 + \beta_1x_1 + \dots$
Sigmoid	Converts a binary score into probability
Softmax	Converts multiple class scores into probabilities
Probability	Estimated chance of each class
Threshold	Converts binary probability into a class
Log loss	Measures how poor the predicted probabilities are

Likelihood	Probability assigned by the model to observed outcomes
Negative log-likelihood	Quantity minimized during training
Gradient	Direction in which cost increases fastest
Gradient descent	Moves coefficients opposite to the gradient
Coefficients	Learned influence of features on log-odds
Solver	Algorithm used to optimize the coefficients
Regularization	Discourages excessively large coefficients

The entire binary logistic regression can be summarized in five equations:

$$z = X\beta$$

$$p = \frac{1}{1 + e^{-z}}$$

★
$$J(\beta) = -\frac{1}{n} \sum [y \log(p) + (1 - y) \log(1 - p)]$$

$$\nabla J = \frac{1}{n} X^T (p - y)$$

$$\beta^{new} = \beta^{old} - \alpha \nabla J$$

That is the central mechanism: **calculate probabilities, measure log loss, adjust coefficients, and repeat until the loss is minimized.**

Binary target → Bernoulli distribution → Logistic model → Likelihood → Log loss → Gradient → Learned coefficients

★ **Binary** means that something has only two possible values or classes.

Bernoulli distribution is a probability model for that binary outcome

Likelihood asks:

For a given set of coefficient values, how well does the model explain the outcomes that we actually observed?

Log odds or logit is same

Loss function

Loss measures the error for **one observation**.

$$L_i = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

For student A:

★
$$L_A = 0.223$$

That is one observation's loss.

Cost function

Cost usually means the average loss across the complete training dataset:

★

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n L_i$$

★ Equivalently: